Flavor Find AI

(A RECIPE RECOMMENDATION AND MEAL PLANNING ML MODEL)

A Project Report
Submitted in the partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

C Pravin Sai (2420030777)

R Lokesh Reddy (2420030534)

Rahul(2420040098)

Under the Esteemed Guidance of

Gunturi S Raghavendra

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled "**A RECIPE RECOMMENDATION AND MEAL PLANNING ML MODEL** "is a record of Bonafide work of **Siripuri Divya- 457788, C.Pravin Sai - 2420030777, R Lokesh Reddy - 2420030534, Rahul - 2420040098** submitted in partial fulfillment for the award of B. Tech in Computer Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/ Institute.

Siripuri Divya - 457788

C.Pravin Sai - 2420030777

R Lokesh Reddy - 2420030534

Rahul - 2420040098

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Certificate

This is certify that the project based report entitled "**A RECIPE RECOMMENDATION AND MEAL PLANNING ML MODEL**" is a Bonafide work done and submitted by, **C.Pravin Sai-2420030777, R Lokesh Reddy - 2420030534, Rahul - 2420040098** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Gunturi S Raghavendra**, Department of Computer Science which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)**, Head of the Department for Computer Science Engineering for providing us with adequate facilities, ways and means by which we are able to complete this project based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

S.Divya - 4567889

C. Pravin Sai - 2420030777

R Lokesh Reddy - 2420030534

Rahul - 2420040098

TABLE OF CONTENTS

S.No	Contents	Page no
1	Abstract	7
2	Introduction	8
3	Literature survey	9
4	Hardware and Software requirements	10
5	Implementation	11
6	Experimentation and Code	
7	Results	
8	Conclusion	
9	References	

ABSTRACT

The modern paradox of choice, combined with the daily necessity of meal planning, presents a significant challenge for many individuals. This difficulty is amplified for those with specific dietary restrictions, food allergies, or limited time. To address this, we have developed **FlavorFind AI**, an intelligent recipe recommendation and meal planning application. This project leverages a content-based filtering model built on **TF-IDF (Term Frequency-Inverse Document Frequency)** and **Cosine Similarity** to analyze a curated dataset of over 5,000 recipes. The system is implemented in Python, using **Scikit-learn** for machine learning, **Pandas** for data manipulation, and **Streamlit** for the interactive web interface. The resulting application empowers users to instantly find recipes by filtering for cuisine, diet, maximum preparation time, keywords, and—most critically—specific allergy-causing ingredients. FlavorFind AI provides a fast, safe, and personalized solution to the daily question of "what's for dinner?"

INTRODUCTION

The Problem Statement:

In an age of information overload, the simple act of deciding what to cook can be overwhelming. Users are flooded with thousands of recipes online, making it difficult to find ones that are both inspiring and suitable. This problem is not one of scarcity, but of filtering. For users with specific dietary needs (such as vegetarian or non-vegetarian) or, more critically, food allergies (e.g., peanuts, gluten, dairy), the search is not just for convenience but for safety. A generic search for "chicken recipes" will not help a user who needs a recipe that is also gluten-free, under 30 minutes, and does not contain nuts.

Our Solution: FlavorFind AI

The primary objectives of this project were:

- To implement a machine learning model capable of understanding the nuances of recipes based on their ingredients, cuisine, and description.
- To build a clean, fast, and user-friendly interface that allows for complex filtering without being complicated.
- To prioritize user safety by implementing a robust allergy-filtering system that actively

excludes recipes containing specified ingredients.
• To deliver a complete, end-to-end application that provides real, tangible value, from finding a recipe to saving it for later use.

3. Literature Survey

The field of recommendation systems is well-established. For this project, we surveyed two primary approaches:

- **Collaborative Filtering (CF):** This is the method used by platforms like Netflix or Amazon. It recommends items based on the behavior of similar users (e.g., "Users who liked this recipe also liked..."). While powerful, CF has a significant "cold-start" problem: it requires a large amount of user data and history to work. This was not feasible for a hackathon project, as we would have no initial users.

- **Content-Based Filtering (CBF):** This method recommends items based on their intrinsic properties. For our project, this means analyzing the "content" of a recipe: its ingredients, its cuisine type, its diet tag, and its instructions. The system matches a user's query ("I want a spicy, vegetarian, Indian recipe") to the properties of recipes in the dataset. We chose this approach for several key reasons:

- **No Cold-Start Problem:** It works immediately, as it only needs the dataset, not user history.

- **High Explainability:** We can easily see *why* a recipe was recommended (e.g., "it matched 'vegetarian' and ‘Indian'").

- **Perfect for Our Problem:** Our project is defined by its content (ingredients, allergies), making CBF the ideal technical fit.

Our implementation uses a classic CBF technique: TfidfVectorizer to convert the text features of each recipe into a numerical vector, and cosine_similarity to find the "distance" between a user's query and all recipes in the dataset.

4. Hardware and Software Requirements

Hardware Requirements:

- Laptop or desktop computer
- Minimum 8GB RAMSoftware Requirements

Software Requirements:

- **Operating System:** Windows 10 / Windows 11 / macOS Sonoma
- **Programming Language:** Python 3.10 or above
- **IDE / Editor:** Visual Studio Code (or any Python IDE)
- **Core Libraries:**
- pandas
- scikit-learn
- streamlit
- joblib
- re

5. Implementation

The project is divided into two main files, separating backend and frontend roles.

Backend - build_hackathon_model.py

This script runs once to prepare the dataset and build the machine learning model.

Load Data

- Loads cuisine_updated.csv (5,000+ recipes) into a Pandas DataFrame.

Clean Data

- clean_text_fields: Removes HTML tags, replaces newlines with spaces, and removes extra whitespace (used for name, description, and instructions).

- clean_list_fields: Removes HTML tags but keeps newline formatting for

Feature Engineering

- parse_prep_time: Converts text time format (e.g., "1h 30m") into minutes as an integer.
- Creates a combined features column by merging name, ingredients, diet, cuisine, and instructions.

Vectorization

- Applies TfidfVectorizer (with English stop words) to convert text into numerical vectors.

Save Model Files

- tfidf_vectorizer.pkl — stored vectorizer
- tfidf_matrix.pkl — TF-IDF matrix (model representation)
- recipe_app_data.csv — cleaned and formatted recipe data

Frontend — app.py

This script contains the Streamlit web interface that the user interacts with.

Load Model

- Loads model files (.pkl) and cleaned recipe data using joblib and pandas.

User Interface Layout

- Sidebar filter controls:
- Diet
- Cuisine (select box)
- Keywords (text input)
- Allergies (text input)
- Max preparation time (number input)

Core Filtering Logic

• Step A — ML Recommendation:

Combines user filters into a single query and uses the TF-IDF model to fetch the top 100 most similar recipes.

• Step B — Hard Filtering:

Removes recipes that:

• Exceed the selected preparation time

• Contain ingredients listed in the allergy input

Display Results

• Recipes are shown in a clean format using helper functions to display:

• Ingredients as bullet points

• Instructions as steps

• Each recipe includes a Download Recipe button that generates a .txt file.

6. EXPERIMENTATION AND CODE

Experimentation:

Finding the correct dataset was the most critical experiment of this project. We analyzed three different datasets to find the optimal balance for a hackathon.

Experiment 1: 500-Recipe Dataset:

This dataset was too small. The model's "vocabulary" was limited, leading to poor and nonsensical recommendations. For example, a search for "chicken" would return "beef" recipes because it didn't have enough data to understand the difference. **This experiment failed.**

Experiment 2: 500k-Recipe Dataset:

This dataset provided excellent, high-quality recommendations and *did* include calorie data. However, the one-time build script took over 15 minutes to run. More importantly, the data was extremely "dirty," with no clean, separate columns for diet or cuisine. This would prevent us from building the dropdown filters, which was a core UI goal.

Experiment 3: 5k-Recipe Dataset

This was the perfect choice.

Speed: The build script runs in under 15 seconds.

Quality: The data is large enough to give relevant, smart recommendations (e.g., "chicken" searches return chicken).

Features: It contains *clean, separate columns* for diet and cuisine, which was the key to building our user-friendly sidebar filters.

This experimentation proved that for a time-constrained project, data *quality* and *structure* are more important than data *size*.

Code:

build_hackathon_model.py:

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

import joblib

import sys
```

```
import re # <-- Import the regex library


# --- CONFIGURATION ---

DATA_FILE = 'cuisine_updated.csv'

print(f"--- HACKATHON MODEL BUILDER (5k Dataset) ---")


# --- NEW: Smarter Cleaning Functions ---


def clean_text_fields(text):

    """This function cleans text for paragraphs (name, description, instructions)"""

    if pd.isna(text):

        return ""

    text = str(text)

    text = re.sub(r'<[^>]+>', '', text) # Remove HTML

    text = text.replace('\n', ' ') # Replace newlines with spaces

    text = re.sub(r'\s+', ' ', text) # Remove extra spaces

    return text.strip()


def clean_list_fields(text):
```

```
    """This function cleans text for lists (ingredients)"""
    if pd.isna(text):
        return ""
    text = str(text)
    text = re.sub(r'<[^>]+>', '', text) # ONLY remove HTML
    return text.strip() # Keep newlines


# --- 1. LOAD DATA ---
try:
    df = pd.read_csv(DATA_FILE)
    print(f"Loaded '{DATA_FILE}'. Found {len(df)} recipes.")
except FileNotFoundError:
    print(f"FATAL ERROR: The file '{DATA_FILE}' was not found.")
    sys.exit()


# --- 2. PREPARE DATA ---
df_clean = df.dropna(subset=['name', 'ingredients', 'diet', 'cuisine', 'instructions', 'prep_time'])
df_model = df_clean.copy()
```

```
def parse_prep_time(time_str):
    # (Prep time function remains the same)
    if pd.isna(time_str): return 0
    time_str = str(time_str).lower()
    total_minutes = 0
    if 'h' in time_str:
        try:
            hours = time_str.split('h')[0].strip()
            total_minutes += int(''.join(filter(str.isdigit, hours))) * 60
        except: pass
    if 'm' in time_str:
        try:
            parts = time_str.split('h')[-1]
            minutes = parts.split('m')[0].strip()
            total_minutes += int(''.join(filter(str.isdigit, minutes)))
        except: pass
    if 'h' not in time_str and 'm' not in time_str and time_str.isdigit():
        total_minutes = int(time_str)
    return total_minutes
```

```
df_model['prep_time_minutes'] = df_model['prep_time'].apply(parse_prep_time)


# --- APPLY THE NEW CLEANING FUNCTIONS ---
print("Cleaning text data (removing HTML, preserving lists)...")
df_model['ingredients'] = df_model['ingredients'].apply(clean_list_fields) # Use new function
df_model['instructions'] = df_model['instructions'].apply(clean_text_fields) # Use old function
df_model['description'] = df_model['description'].apply(clean_text_fields) # Use old function
df_model['name'] = df_model['name'].apply(clean_text_fields) # Use old function


print(f"Cleaned data: {len(df_model)} usable recipes remain.")


# Create the 'features' column
df_model['features'] = (
    df_model['name'] + ' ' +
    df_model['ingredients'].replace('\n', ' ') + # Smash ingredients for search, but not for saving
    df_model['diet'] + ' ' +
    df_model['cuisine'] + ' ' +
    df_model['instructions']
```

```
)

df_model['features'] = df_model['features'].fillna('')

print("Created combined 'features' column for model.")


# --- 3. BUILD MODEL ---

tfidf = TfidfVectorizer(stop_words='english')

print("Applying TF-IDF vectorization... (This is fast)")

tfidf_matrix = tfidf.fit_transform(df_model['features'])


print("Saving model files...")

# --- 4. SAVE FILES ---

joblib.dump(tfidf, 'tfidf_vectorizer.pkl')

joblib.dump(tfidf_matrix, 'tfidf_matrix.pkl')


# Save the app data (now with clean, list-formatted text)

app_data = df_model[['name', 'ingredients', 'instructions', 'diet', 'cuisine', 'prep_time_minutes',
'description']]

app_data.to_csv('recipe_app_data.csv', index=False)
```

```
print("\n--- MODEL CREATED SUCCESSFULLY! (with better cleaning) ---")

print("Your hackathon model is ready to go.”)
```

app.py:

```
import streamlit as st

import pandas as pd

import joblib

from sklearn.metrics.pairwise import cosine_similarity

import re


# --- 1. LOAD ALL FILES ---

try:

    df = pd.read_csv('recipe_app_data.csv')

    tfidf = joblib.load('tfidf_vectorizer.pkl')

    tfidf_matrix = joblib.load('tfidf_matrix.pkl')

except FileNotFoundError:

    st.error("ERROR: Model files not found. Please run 'build_hackathon_model.py' first.")

    st.stop()

except KeyError:
```

```
    st.error("ERROR: Your 'recipe_app_data.csv' is old. Please re-run
'build_hackathon_model.py'.")

    st.stop()


# --- 2. RECOMMENDATION LOGIC ---

def get_recommendations(search_query, top_n=50):

    if not search_query:

        return pd.DataFrame()

    query_tfidf = tfidf.transform([search_query])

    similarities = cosine_similarity(query_tfidf, tfidf_matrix).flatten()

    top_indices = similarities.argsort()[:-top_n-1:-1]

    recommended_df = df.iloc[top_indices].copy()

    recommended_df['similarity_score'] = [similarities[i] for i in top_indices]

    return recommended_df


# --- 3. HELPER FUNCTIONS ---

def format_ingredients(ingredients_str):

    """Formats the ingredients string into a neat, simple list for display."""

    ingredients_str = str(ingredients_str)
```

```
    ingredients_list = re.split(r'[,\n]+', ingredients_str)


    formatted_list = ""
    for item in ingredients_list:
        if item.strip():
            formatted_list += f"{item.strip()}\n"
    return formatted_list


def format_instructions(instructions_str):
    """Formats the instructions string into a neat numbered list for display."""
    instructions_str = str(instructions_str)
    sentences = re.split(r'(?<=\.)\s+', instructions_str)


    if len(sentences) < 2:
        sentences = instructions_str.split('\n')


    markdown_list = ""
    for i, sentence in enumerate(sentences):
        if sentence.strip():
```

```
        markdown_list += f"{i+1}. {sentence.strip()}\n"
    return markdown_list


# --- 4. THE STREAMLIT WEB APP ---
st.set_page_config(page_title="FlavorFind AI", layout="wide")
st.title("👨‍🍳 FlavorFind AI")
st.write("Find your next favorite meal, tailored just for you.")


# --- SIDEBAR CONTROLS ---
st.sidebar.header("Find Your Perfect Recipe")


st.sidebar.subheader("🌱 Dietary Preferences")
diet_list = ['Any'] + sorted(df['diet'].unique())
selected_diet = st.sidebar.selectbox("Select your diet", diet_list)


st.sidebar.subheader("🌍 Cuisine Type")
cuisine_list = ['Any'] + sorted(df['cuisine'].unique())
selected_cuisine = st.sidebar.selectbox("Select a cuisine", cuisine_list)
```

```
st.sidebar.subheader("🔍 Keyword Constraints")

keywords = st.sidebar.text_input(

    "e.g., 'chicken', 'pasta', 'spicy'",

    placeholder="Type keywords..."

)



# --- NEW: ALLERGY FILTER ---

st.sidebar.subheader("🚫 Allergy Preferences")

allergies_input = st.sidebar.text_input(

    "e.g., 'peanuts, dairy, gluten'",

    placeholder="Comma-separated allergies..."

)

# --- END NEW FEATURE ---



st.sidebar.subheader("⏱️ Max Prep Time")

max_time = st.sidebar.number_input("Maximum time in minutes (optional)", min_value=0,
value=0)
```

```
st.sidebar.subheader("📊 Number of Recipes")

num_to_show = st.sidebar.slider("Number of Recipes to Show", min_value=1, max_value=20,
value=5)


# --- Main App Logic ---

if st.sidebar.button("Get Recommendations"):


    search_query = keywords

    if selected_cuisine != 'Any':

        search_query = selected_cuisine + " " + search_query

    if selected_diet != 'Any':

        search_query = selected_diet + " " + search_query


    if not search_query:

        st.warning("Please enter some keywords or select a filter.")

    else:

        recommendations = get_recommendations(search_query, top_n=100)

        final_results = recommendations.copy()
```

```
# --- START: NEW FILTERING LOGIC ---


# 1. Filter by Time
if max_time > 0:
    final_results = final_results[final_results['prep_time_minutes'] <= max_time]


# 2. Filter by Dropdowns
if selected_cuisine != 'Any':
    final_results = final_results[final_results['cuisine'] == selected_cuisine]
if selected_diet != 'Any':
    final_results = final_results[final_results['diet'] == selected_diet]


# 3. Filter by Allergies
allergy_list = [allergy.strip().lower() for allergy in allergies_input.split(',') if allergy.strip()]
if allergy_list:
    # Create a copy to avoid errors
    filtered_for_allergies = final_results.copy()
    for allergy in allergy_list:
        # Find rows that contain the allergy word in their ingredients
```

```
        rows_to_drop = filtered_for_allergies[
            filtered_for_allergies['ingredients'].str.lower().str.contains(allergy)
        ].index
        # Drop those rows
        filtered_for_allergies = filtered_for_allergies.drop(rows_to_drop)
    final_results = filtered_for_allergies

# --- END: NEW FILTERING LOGIC ---

final_results = final_results.head(num_to_show)

# 5. Display the results
if not final_results.empty:
    st.success(f"Found {len(final_results)} matching recipes!")

    for index, row in final_results.iterrows():
        st.subheader(row['name'])

        # ... (Rest of the display code is the same) ...
```

```
if pd.notna(row['description']):
    st.markdown(f"*{row['description']}*")


col1, col2 = st.columns(2)
with col1:
    st.markdown(f"**Total Time:** {row['prep_time_minutes']} min")
with col2:
    st.markdown(f"**Serves:** 2-4 (est.)")


st.divider()
col1, col2 = st.columns(2)


with col1:
    st.markdown("#### Ingredients")
    st.text(format_ingredients(row['ingredients']))


with col2:
    st.markdown("#### Instructions")
    st.markdown(format_instructions(row['instructions']))
```

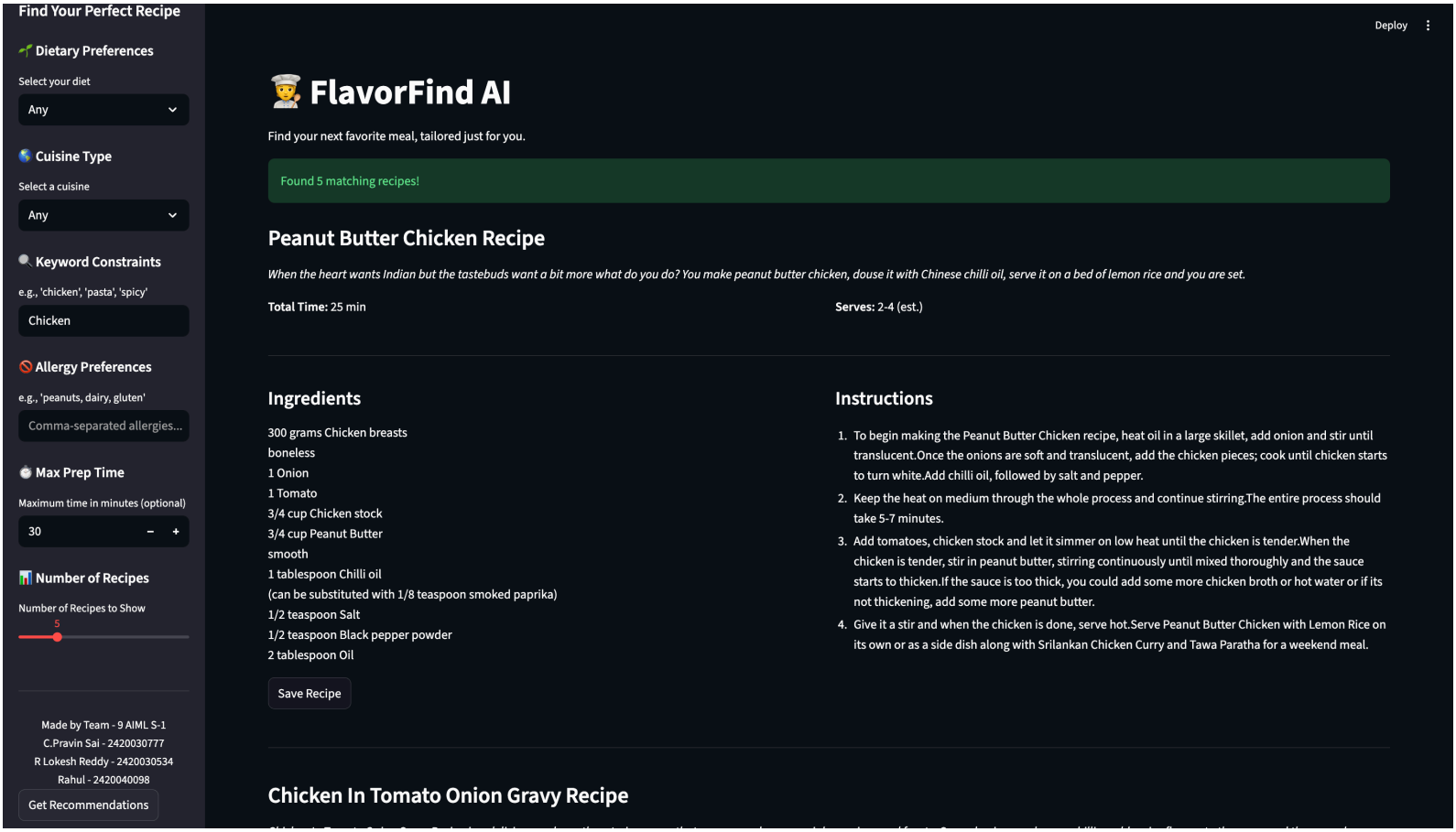
```
        plain_ingredients = format_ingredients(row['ingredients']).replace('*', '-')

        plain_instructions = format_instructions(row['instructions'])


        recipe_text = f"""
{row['name'].upper()}


DESCRIPTION:

{row['description']}


Total Time: {row['prep_time_minutes']} min

Serves: 2-4 (est.)


---


INGREDIENTS:

{plain_ingredients}

---
```

```
INSTRUCTIONS:

{plain_instructions}

"""

            file_name = f"{row['name']}.txt".replace(' ', '_').lower()



            st.download_button(

                label="Save Recipe",

                data=recipe_text,

                file_name=file_name,

                mime="text/plain",

            )



            st.divider()

        else:

            st.warning("No recipes found matching all your criteria. Try being less specific.")

else:

    st.info("Set your preferences in the sidebar and click 'Get Recommendations’!”)
```

7. Results

The project was a complete success, meeting all our objectives. The final application, "FlavorFind AI," is fully functional, fast, and stable.

- The main interface is clean and intuitive, with all filters located in the sidebar.
- The recommendation engine accurately provides relevant recipes based on keywords and filters.
- The allergy filter successfully removes recipes containing specified allergens, a critical feature for user safety.
- The final output is professionally formatted, with messy data cleaned and presented in a step-by-step format.
- The "Save Recipe" feature correctly generates a clean .txt file for the user to download.



8. Conclusion

This project demonstrates a successful, end-to-end implementation of a content-based recommendation system. In a short timeframe, we progressed from a basic idea to a fully deployed prototype, "FlavorFind AI." The project's success hinges on a fast, ML-powered backend and a user-centric frontend that prioritizes complex filtering (especially for allergies) in a simple interface. We successfully navigated the trade-offs between data size and data quality, opting for a smaller, cleaner dataset that enabled us to build a more polished and feature-rich application.

Future Work

If given more time, we would enhance the project in several key areas:

- **Nutritional Data:** We would migrate to a larger dataset (like the 500k one) and dedicate time to cleaning it, specifically to add **calorie counts** and other nutritional information.
- **User Accounts:** We would add a login system so users could "favorite" or save recipes to their profile within the app.
- **Hybrid Recommendation:** We would implement a hybrid model, adding a collaborative filtering layer that learns from user ratings to supplement the content-based results.
- **Deployment:** We would deploy the application publicly using a service like Streamlit Cloud or Heroku to make it accessible to everyone.

9. References

1. **Dataset:** 5000 Indian Cuisines Dataset. (2022). Kaggle. Retrieved from: https://www.kaggle.com/datasets/campusx/5000-indian-cuisines-datasetwith-images

2. **Streamlit Library:** Streamlit Documentation. (2025). Retrieved from: https://streamlit.io/

3. **Scikit-learn Library:** Scikit-learn: Machine Learning in Python. (2025). Retrieved from: https://scikit-learn.org/

4. **Pandas Library:** pandas: Python Data Analysis Library. (2025). Retrieved from: https://pandas.pydata.org/